Notification Platform — PRD v2.0

Oct 9, 2026 · @Gerick

What changed from v1.0

v1.0 had the right ingredients; v2.0 closes nine gaps that a reviewer or interviewer would probe first. I have not seen your existing codebase, so every decision below is a target, and Phase 0 maps each one to what already exists.

#	Gap in v1.0	Fix in v2.0
1	"Accepted" meant a Kafka publish succeeded, so a broker outage would reject or lose requests	Transactional outbox: 202 means the request is durable in PostgreSQL; Kafka publication is decoupled
2	No Kafka design (topics, keys, retries, dead letters, message contents)	Topology, retry tiers, dead-letter handling, and a claim-check message that carries no personal data
3	Database left open ("PostgreSQL or existing")	PostgreSQL, with a schema, partitioning and retention plan
4	Authentication described as "an appropriate mechanism"	Keycloak: OAuth 2.0 client credentials for client apps, OIDC with a backend-for-frontend for the dashboard, defined roles
5	Capacity defined in principle only	A repeatable method: open-model k6 executors, staircase search, 30-minute confirmation, 30% headroom, CI regression gates
6	No privacy or email-standards requirements	Data minimization, retention, suppression list, SPF/DKIM/DMARC checks, one-click unsubscribe (RFC 8058)
7	No SLOs or telemetry design	SLIs and SLOs, OpenTelemetry, Prometheus, alert rules, runbooks

#	Gap in v1.0	Fix in v2.0
8	Free-tool promise never audited	License audit: MediatR, AutoMapper, MassTransit 9 and FluentAssertions 8 moved to commercial or reciprocal licenses and are excluded
9	Five phases with no baseline, priorities or portfolio plan	Phase 0 baseline on your existing code, MoSCoW priorities, and a portfolio packaging section

Kept from v1.0: the four-service split, k6 as the generator, SignalR for live updates, Docker Compose for local runs, and the rule that capacity is the highest *sustained* rate, never the highest observed rate.

Product overview

Notification Platform is a multi-tenant email service: client applications call one authenticated HTTP endpoint, and the platform queues, renders, sends, tracks and reports on every message. It is built to be operated, not just demonstrated, so each requirement below has a way to verify it.

Goals

- Never lose an accepted notification, and tell the caller honestly what has and has not happened to it.
- Give operators a dashboard that explains where every message is and why it failed.
- Be safe by default: authenticated clients, no personal data in logs or queues, no way to use the platform as a spam relay.
- Measure capacity with a repeatable method instead of quoting a peak number.
- Run entirely on free and open-source software.

Personas

Persona	Needs	Main surfaces
Client developer	Integrate quickly, get clear errors, see why a message failed	Send API, OpenAPI docs, status endpoint
Platform admin	Manage clients, templates, sender profiles and quotas	Dashboard admin modules

Persona	Needs	Main surfaces
Operator	Watch health, retry failures, run load tests	Overview, notification explorer, load tests
Viewer or auditor	Read-only access with recipient data masked	Dashboard (read-only), audit log

Out of scope for v2.0

- Marketing campaigns, audience lists and A/B tests. The platform is transactional; marketing needs consent management it does not provide.
- Attachments, inbound email, SMS and push. The channel layer stays replaceable so these can be added later.
- Multi-region active-active deployment.
- A drag-and-drop template designer. Templates are code-edited with live preview.
- Load tests that send real email, and load tests against production by default.

Success criteria

Outcome	Target	Verified by
Accepted notifications lost	0, including broker, database and worker restarts	Chaos test suite
Accept latency	p95 < 200 ms and p99 < 500 ms at the recommended operating rate	k6 capacity run
Time from accept to SMTP hand-off	99% within 30 s at the recommended operating rate	k6 run against the mock SMTP server
Duplicate sends after forced worker crashes	Measured and documented; target < 0.1%	Chaos test suite
Sustainable accept and processing rates	Published with method, hardware and headroom	Capacity report
Time from clone to working demo	Under 5 minutes, one command	Fresh-machine run documented in the README

Key decisions

v1.0 left these decisions open or implicit; each one below becomes an architecture decision record (ADR) in the repo.

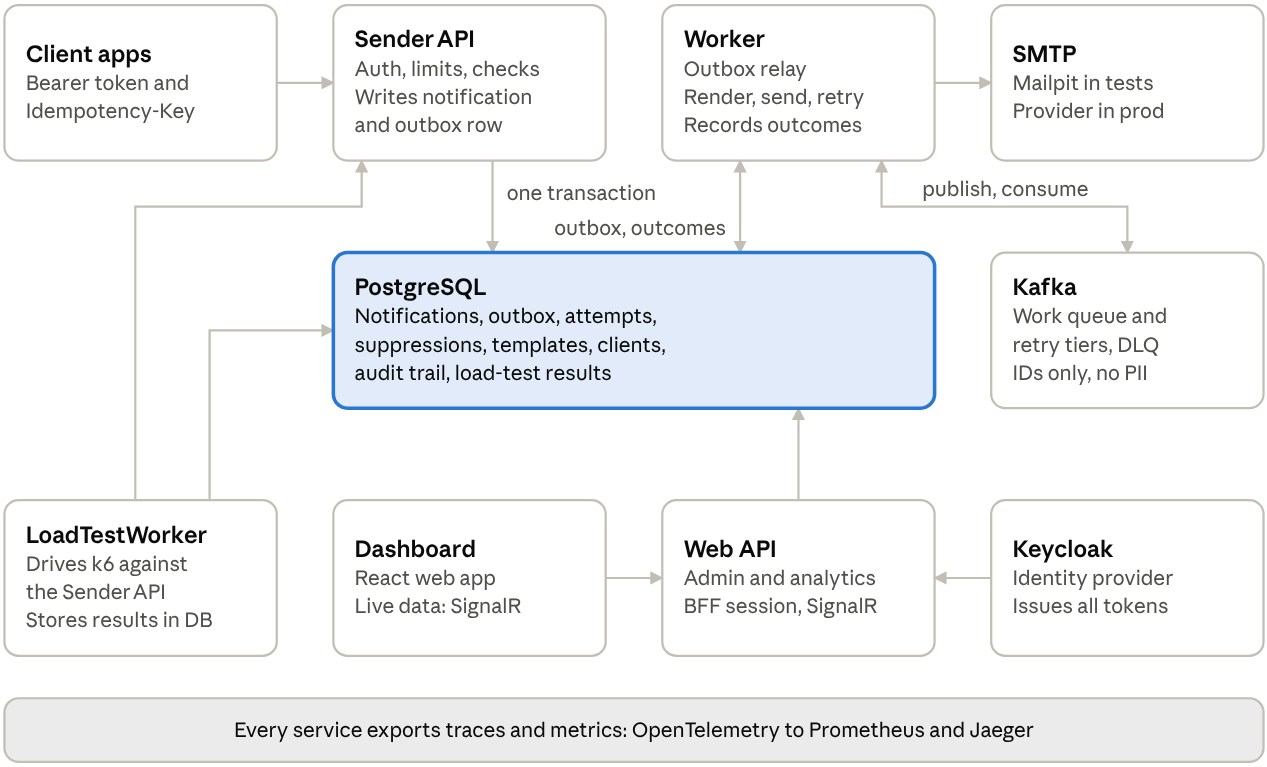
#	Decision	Choice	Why
1	What "accepted" means	Transactional outbox: the notification row and an outbox row commit in one PostgreSQL transaction, then the API returns 202	Accepting does not depend on Kafka being up, and nothing accepted can be lost between database and broker
2	Delivery semantics	At-least-once processing with idempotent state transitions; no exactly-once claim	Kafka transactions cannot include an SMTP send, so duplicates are bounded and measured rather than denied
3	Kafka client	Confluent.Kafka (Apache 2.0) used directly inside .NET hosted services	Full control of offset commits and partition pausing; avoids MassTransit 9, which requires a license
4	Message contents	Claim check: messages carry identifiers and routing data; the worker loads the encrypted payload from PostgreSQL	No personal data in Kafka or dead-letter topics, and small messages
5	Retries	Non-blocking retry topics with delay tiers, then a dead-letter topic	A failing message never blocks its partition
6	Database	PostgreSQL; EF Core for writes, hand-written SQL for dashboard aggregates	One store for state, outbox and analytics keeps operations simple
7	Authentication	Keycloak: client credentials for apps; Authorization Code with PKCE through a backend-for-frontend for the dashboard	Standards-based, free, and no tokens in browser storage

#	Decision	Choice	Why
8	Template engine	Fluid (Liquid) in a sandbox, not Razor	Liquid cannot execute arbitrary code; Razor templates can
9	SMTP	MailKit; Mailpit as the mock server for development, CI and load tests	Mature and free; real email is never sent during tests
10	Load generator	k6 CLI with open-model executors and scripts from a versioned template library	Arrival-rate executors hold offered load constant when the system slows down
11	Live updates	SignalR pushes aggregated snapshots every 1 to 2 seconds	Cheap and bounded, with polling as the fallback
12	Application structure	Plain handler classes and hand-written mapping; no mediator or mapping library	Removes two license-restricted libraries and keeps the code easy to read
13	API versioning	Keep POST /api/v1.1/Email/SendEmail for existing callers; add Idempotency-Key and problem-details errors; change the version segment only for breaking changes	Existing consumers keep working and changes stay additive
14	Repository	One monorepo with one solution, one frontend folder and one compose file	Simplest for a single maintainer, and CI sees everything

Architecture

Four .NET services and a React dashboard share one PostgreSQL database and one Kafka cluster. The Sender API never talks to Kafka, which is why a broker outage cannot stop acceptance.

An accepted request is durable in PostgreSQL before Kafka is involved



architecture · 4 services, 2 stores, 1 SMTP target

Accepting a request needs only the Sender API and PostgreSQL. The worker's outbox relay moves rows to Kafka afterwards, and its consumers write each outcome back to PostgreSQL.

Components

Component	Responsibility	Scales by
NotificationSenderApi	Authenticate, validate, rate limit; write the notification and its outbox row in one transaction; return 202	Replicas behind a load balancer
NotificationWebApi	Admin CRUD, dashboard analytics, backend-for-frontend session, SignalR hub, load-test run API	Stateless replicas

Component	Responsibility	Scales by
NotificationWorker	Hosts the outbox relay and the Kafka consumers: render, send, retry, record outcomes	Consumers up to the partition count; the relay can be split out later
LoadTestWorker	Claims runs, starts k6, aggregates metrics, stores results	One active run by default
Dashboard	React single-page app served as static files	Any web server or CDN
Lib	Contracts, DTOs, enums and common abstractions only; no orchestration and no dependency on application or load-test code	Not applicable

The NotificationWorker is a separate Worker Service from day one. v1.0 allowed hosting it inside the Sender API, which would put Kafka and SMTP failures in the API's process.

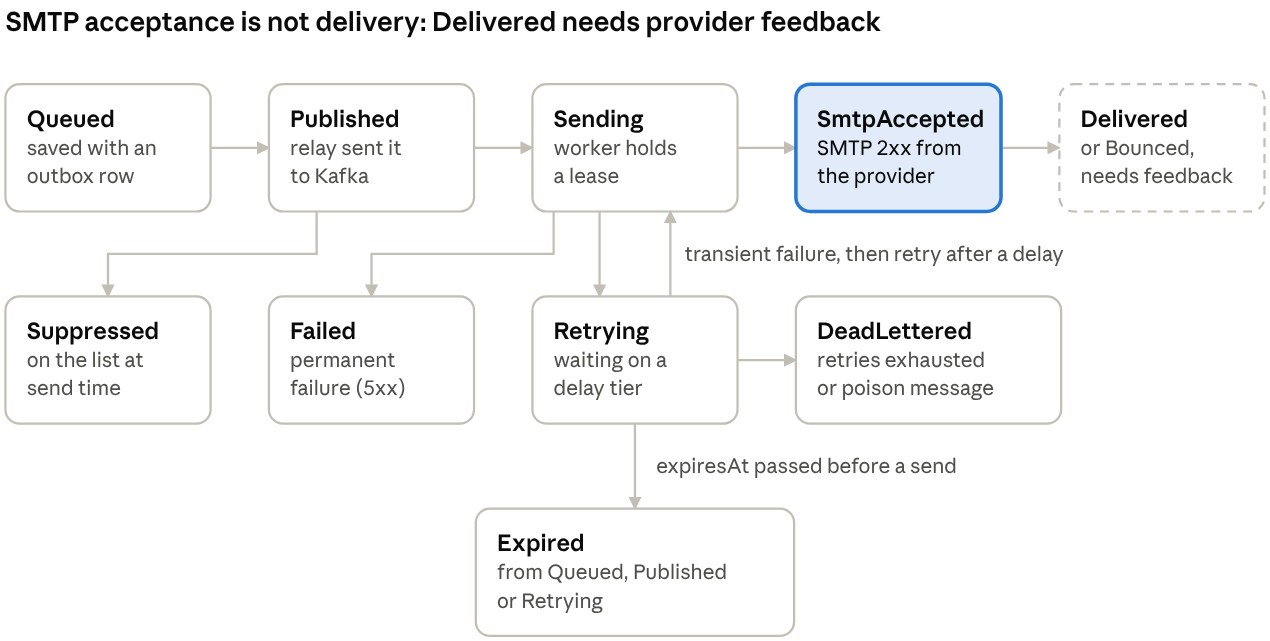
Kafka topology

Topic	Purpose	Key	Partitions (start)	Retention
notifications.transactional	Primary work queue	notificationId	6	3 days
notifications.retry.1m, .5m, .30m, .2h	Delay tiers	notificationId	3 each	3 days
notifications.dlq	Exhausted and poison messages	notificationId	3	14 days

- Partition counts are starting values; the capacity tests set the real numbers. A separate `notifications.bulk` topic for lower-priority mail is a Could.
- Messages carry `schemaVersion`, `notificationId`, `correlationId`, `clientId`, `attempt`, `notBefore` and a W3C `traceparent` header. They never carry recipients or template data.

- The producer is idempotent with acks=all. Consumers disable auto-commit and use the cooperative-sticky assignor so a rebalance does not stop the whole group.
- Keying by notification ID spreads load evenly. Ordering between notifications is not needed.

Notification lifecycle



notification states · 1 main path, 1 retry loop, 4 failure exits

Final states are SmtpAccepted, Delivered, Bounced, Failed, DeadLettered, Suppressed and Expired. Every transition is a compare-and-set in PostgreSQL, so a redelivered message cannot move a notification backwards.

Email API and pipeline

The public API has one job: authenticate the caller, validate the request, store it durably, and answer 202 with an identifier. Everything after that is asynchronous.

Endpoint

`POST /api/v1.1/Email/SendEmail` with `Authorization: Bearer <token>` and a required `Idempotency-Key` header.

```
{
  "to": ["customer@example.com"],
  "templateId": "invoice-created",
  "data": { "invoiceNumber": "INV-1001" },
  "expiresAt": "2026-10-09T10:30:00Z"
}
```

Response, `202 Accepted` with a `Location` header:

```
{
  "notificationId": "0198a3f2-7c1e-7b2a-9d4e-5f6a7b8c9d0e",
  "status": "Queued",
  "correlationId": "b7f1c2e4-3a5d-4e6f-8a9b-0c1d2e3f4a5b"
}
```

- Identity comes from the token. If a v1.0 caller still sends `clientId`, it must match the token's client or the call fails with 403.
- `to` takes 1 to 50 recipients. `data` is limited to 64 KB and the whole body to 256 KB.
- `expiresAt` is optional. A notification not handed to SMTP by then ends as Expired, which suits one-time codes.
- Callers check progress with GET /api/v1/notifications/{id}, which returns only their own notifications. Attachments are out of scope for v2.0.

Request processing order

1. Validate the token (issuer, audience, signature, lifetime) and map it to an active client record.
2. Apply the client's rate limit; reject with 429 and `Retry-After`.
3. Validate the body: size, recipient syntax and count, template access, and `data` against the template version's JSON Schema.
4. Check idempotency: the same key with the same payload hash returns the original response; the same key with a different payload returns 409.
5. Check the client's daily quota.
6. In one transaction, insert the notification as Queued and insert its outbox row.
7. Return 202.

Error contract

Status	When
400	Malformed JSON or missing Idempotency-Key
401	Missing, expired or invalid token
403	Inactive client, template not granted, or clientId does not match the token
404	Unknown template
409	Idempotency key reused with a different payload
413	Body larger than 256 KB
422	Validation failed, with a per-field error list
429	Rate limit or daily quota exceeded, with Retry-After
503	Database unavailable; retry with the same Idempotency-Key

Every error body is an RFC 9457 problem-details document with a stable `type` URI and the correlation ID. A replayed request returns the original body plus `Idempotent-Replayed: true`; keys stay valid for 24 hours by default.

Rate limiting and quotas

Acceptance and processing are limited separately, as v1.0 required.

- Acceptance: the ASP.NET Core rate limiter (token bucket) partitioned by client, with limits read from the client record. It runs in-process for one API instance; with several replicas it moves to a shared store (Valkey, BSD-3) behind the same interface.
- Processing: the worker throttles sends per client and per sender profile so one client cannot monopolize a provider.
- Quota: a daily send count per client, kept in PostgreSQL.

Templates

- Versions are immutable once published and move through Draft, Published and Deprecated.
- Each version has a subject, an HTML body, a required plain-text body (sent as multipart/alternative) and a JSON Schema for its variables.

- Missing or mistyped variables fail at accept time with 422, not at send time.
- Fluid runs sandboxed: HTML auto-escaping on, allow-listed filters, no .NET type access, and caps on render time and output size.
- Each template has a category, transactional or subscription. Subscription mail carries `List-Unsubscribe` headers and honors the suppression list.

SMTP sending

- MailKit with TLS required (STARTTLS or implicit); plain SMTP only in the development profile.
- A sender profile binds a from-address, SMTP host and credentials; a client can only use the profiles assigned to it.
- Timeouts default to 10 s to connect and 30 s to send, and are configurable.
- SMTP 2xx is accepted, 4xx and network errors are transient, 5xx is permanent.
- Each message gets a Message-ID derived from the notification ID, so duplicates can be recognized downstream.
- Headers are set through MailKit's API, never by string concatenation, and CR or LF in recipients or subjects is rejected.

Suppression list

Addresses that must not be mailed (hard bounces, complaints, unsubscribes, manual entries) are stored as keyed hashes. The worker checks the list at send time, and a suppressed recipient ends the notification as Suppressed, which is a normal outcome rather than a failure.

Reliability and data consistency

The platform guarantees that an accepted notification is never lost and is processed at least once. It does not guarantee exactly-once delivery, because an SMTP send cannot take part in a database or Kafka transaction.

Guarantees and how they are met

Guarantee	Mechanism
Accepted means durable	The notification and outbox rows commit in one PostgreSQL transaction before 202 is returned

Guarantee	Mechanism
Durable means published	The outbox relay uses an idempotent producer with acks=all and marks a row published only after the broker acknowledges it; several relays can run safely by locking rows and skipping rows already locked
Published means processed	Consumers disable auto-commit and commit an offset only after the outcome is recorded in PostgreSQL
Redelivery is harmless	State changes are compare-and-set, terminal notifications are skipped, and a send in progress holds a time-limited lease
Duplicates are bounded	The only duplicate window is a crash between SMTP acceptance and the database write; it lasts at most one lease period and is measured in the chaos suite

Failure classes and retry policy

Class	Examples	Action
Transient	SMTP 4xx, timeout, connection refused, temporary DNS failure	Retry through the next delay tier
Permanent	SMTP 5xx, invalid recipient, template render error	Fail immediately, no retry
Poison	Message cannot be read, or names an unknown notification	Dead-letter immediately
Expired	`expiresAt` passed before the send	Mark Expired, no retry

- Retry tiers wait 1 minute, 5 minutes, 30 minutes and 2 hours, each with full jitter, for at most 5 attempts in total (configurable). After the last attempt the notification is dead-lettered.
- A retry consumer pauses its partition until the message's `notBefore` time, so waiting never blocks other topics.

- When an SMTP profile's circuit breaker (Polly) opens, consumers for that profile pause instead of burning retry attempts, and resume when a probe succeeds.
- Operators can inspect and replay dead letters from the dashboard. A replay needs a reason, is rate-limited and is written to the audit log.
- Each worker bounds its in-flight sends. Consumer lag is the scaling signal, and consumers scale up to the partition count.
- On SIGTERM a worker stops polling, finishes in-flight sends within 30 seconds, commits offsets and releases leases.

Expected behavior under failure

Each row is an automated test using container restarts and Toxiproxy fault injection.

Failure	Expected behavior
Kafka unavailable at accept time	API still returns 202; outbox rows accumulate and drain after recovery; an alert fires on outbox age
PostgreSQL unavailable	API returns 503; workers pause; nothing is acknowledged that was not recorded
SMTP provider down	Breaker opens, consumers pause, retry attempts are preserved, processing resumes on recovery
Worker killed mid-send	Lease expires, message is redelivered, a possible duplicate is recorded as such
Poison message	Dead-lettered with a reason; the partition keeps moving
Relay killed mid-batch	Unpublished rows remain and are republished; consumers deduplicate by state
Caller repeats a request	Idempotency returns the original notification ID; no second notification exists
Consumer rebalance mid-processing	Uncommitted work is redelivered and handled as above

Dashboard

The dashboard is a React single-page app for operators and admins. Its central rule: HTTP acceptance, Kafka publication, SMTP acceptance and final delivery are four separate numbers, never one "success rate".

Modules

Module	What it shows or does	Minimum role
Overview	Tiles for accepted, queued, in flight, SMTP-accepted, failed, dead-lettered, suppressed and expired; throughput; accept, queue-wait and send latency at p50, p95 and p99; outbox age; consumer lag per topic; dead-letter depth; service health; recent errors	Viewer
Notification explorer	Server-side search, filter and paging by ID, correlation ID, client, status, date and recipient; a detail page with the attempt timeline, failure category, sanitized SMTP response and retry history; retry and replay for eligible items	Viewer to read, Operator to act
Clients	Create, edit, deactivate; rate limit and daily quota; template grants and sender profiles; usage and error charts; credential rotation with the secret shown once	Admin
Templates	Code editor with Liquid highlighting, variable schema, live preview with sample data in a sandboxed frame, version history with diff, publish and deprecate, test send to an allow-listed mailbox	Admin
Sender profiles	SMTP host, port, TLS mode and write-only credentials; connectivity test; credential rotation; last failure category; sender-domain check for SPF, DKIM and DMARC	Admin

Module	What it shows or does	Minimum role
Suppression list	Search, add and remove with a reason; CSV import and export	Operator
Load tests	Configure, run, watch live, cancel, compare, pin baselines, read capacity reports	Viewer to read, LoadTester to run
Audit log	Who changed what and when, filterable and exportable	Admin

Users and roles are managed in Keycloak. The dashboard shows the signed-in user's roles but does not manage accounts.

Real-time behavior

- The server pushes aggregated snapshots every 1 to 2 seconds over SignalR, never one event per notification.
- If the socket drops, the client falls back to polling every 5 seconds and shows the connection state and the age of the data.
- Every time-series view has a range picker (15 minutes, 1 hour, 24 hours, 7 days) and says whether it is live or historical.

Frontend stack and conventions

- React with TypeScript in strict mode, built with Vite.
- TanStack Query for server state, TanStack Table for grids, React Hook Form with Zod for forms, Recharts for charts, Tailwind with shadcn/ui for components.
- The API client and its types are generated from the OpenAPI document in CI, so the UI cannot drift from the API.
- Sign-in uses a backend-for-frontend session cookie (HttpOnly, Secure, SameSite=Strict). No tokens live in browser storage.

UX and accessibility requirements

- WCAG 2.2 AA: keyboard operation, visible focus, sufficient contrast, and a data-table alternative for every chart.
- Every view has loading, empty and error states.
- Recipient addresses are masked by default; unmasking needs a separate permission and is audited.

- Dark and light themes; the layout targets 1024 px and wider and stays usable on tablets.
- Initial JavaScript stays under 300 KB gzipped, enforced by a CI bundle-size check.

Load testing and capacity

The load-testing subsystem answers one question with evidence: what is the highest rate each stage of the pipeline can sustain while meeting its objectives, and what limits it first?

Principles

- Open workload model. Tests use k6's constant-arrival-rate and ramping-arrival-rate executors, which start requests at a fixed rate whether or not earlier ones have finished. A closed model (a fixed pool of virtual users) slows down when the system slows down and hides overload, so it is used only for smoke tests.
- A test is invalid if the generator is the bottleneck: k6 reports dropped iterations, or generator CPU stays above 80%. Invalid runs are marked Incomplete, never Passed.
- k6 runs apart from the system under test, under its own resource limits, and is sized for the target rate.
- Email goes to Mailpit, never to a real provider. An optional fault profile adds latency and failures at the SMTP stage so retry behavior can be load tested too.

Test types

Shapes below are defaults and are configurable within the caps.

Type	Purpose	Default shape	Passes when
Smoke	Verify scenario, credentials and checks	1 request per second for 1 minute	Every check passes
Load	Confirm behavior at expected peak	Ramp to the target rate, hold 15 minutes	All thresholds hold during the hold
Stress (breakpoint)	Find where and why the system degrades	Raise the rate 10% every 5 minutes until a threshold fails	The breakpoint and the saturated resource are identified

Type	Purpose	Default shape	Passes when
Spike	Test burst handling and recovery	Jump to 10 times the operating rate within 10 seconds, hold 2 minutes, return	Errors stay within budget and lag returns to baseline within 5 minutes
Soak	Find leaks and slow degradation	70% of capacity for 2 hours (up to 8 on request)	No steady growth in memory, lag or latency
Capacity search	Produce the sustainable rate	Staircase plus confirmation hold, below	The confirmation hold passes

How capacity is defined

Capacity is the highest rate that passes every threshold for a sustained hold, not the highest rate observed.

1. Run a staircase of arrival rates. Each step has a 1-minute warm-up and a hold of at least 5 minutes.
2. A step passes only if, during its hold, the HTTP error rate is below 1%, p95 accept latency is below 200 ms, no iterations were dropped, outbox age and consumer lag are flat or falling, and CPU, memory and database connections are not exhausted.
3. The first failing step ends the staircase. The highest passing step is the candidate.
4. Hold the candidate for 30 minutes. If it fails, drop one step and repeat. If it passes, it is the sustainable rate.
5. Drop to 50% of that rate and confirm that lag drains to baseline within 5 minutes.
6. Publish the recommended operating rate as 70% of sustainable capacity.

Thresholds are configurable per profile; the values above are the defaults.

Three capacities are reported separately because they have different limits:

- API acceptance rate.
- Notification processing rate, measured against the mock SMTP server.
- SMTP provider rate, taken from the provider's documented limits and set as a cap, never load tested against a real provider.

Each report also names the bottleneck: the first resource to saturate, such as API CPU, database connections or write latency, Kafka partitions, worker concurrency or SMTP latency.

Execution flow

1. An authorized user creates a test configuration in the dashboard.
2. NotificationWebApi validates it against the caps and the environment allow-list and stores a run as Queued.
3. LoadTestWorker claims the run with a database row lock; one run executes at a time by default.
4. The worker renders a k6 script from a versioned template, passes parameters and secrets as environment variables, and starts k6 as a child process.
5. The worker reads k6′s metric output, computes 1-second windows, stores 5-second aggregates and pushes live snapshots over SignalR.
6. In parallel it samples backend metrics from Prometheus: outbox age, consumer lag, processing rate, CPU, memory and database connections.
7. On finish it stores the summary and a verdict (Passed, Failed, Cancelled or Incomplete) with per-threshold results.
8. Cancelling sends SIGTERM to k6, then SIGKILL after a grace period, and cleans up. A worker that loses its heartbeat leaves the run Incomplete, and a reaper job closes orphaned runs.

How the worker reads k6′s metrics is settled by a short spike in Phase 3 and recorded as an ADR. The candidates are tailing k6′s JSON output, k6′s Prometheus remote-write output, and polling k6′s local REST API. The deciding criteria are generator overhead at the target rate and the accuracy of windowed percentiles. Start with the JSON output and switch only if the spike shows measurable overhead.

Safety controls

Control	Requirement
Target allow-list	Only pre-approved environments (host and port) can be targeted; resolved addresses are checked at start so a DNS change cannot redirect a test; the default allow-list is local and staging

Control	Requirement
Production	Disabled by default. If enabled: a feature flag, a second approver, a scheduled window, lower caps, automatic abort above a 2% error rate, and full audit
Hard caps	Maximum rate, duration, virtual users and payload size per environment, enforced on the server and not only in the UI
No arbitrary execution	No user-supplied scripts, shell commands or CLI arguments; users pick a scenario and fill validated parameters
Secrets	Injected as environment variables at run time; never written to disk, logs or results; k6 output is redacted
Isolation	k6 runs as a separate process under container CPU and memory limits, as a non-root user, with network access to allow-listed targets only; no container-runtime socket is mounted into the worker
Concurrency	One run at a time by default; others wait in the queue
No real email	Load tests use a synthetic client flagged as such, and the platform lets synthetic clients use only the mock SMTP profile
Visibility	Test traffic is tagged with the run ID and excluded from business analytics by default
Authorization	Role LoadTester required; the initiator is recorded; every start, cancel and baseline change is audited

Metrics collected

k6 cannot see inside the backend, so server-side metrics depend on the instrumentation described under Observability and SLOs.

Metric	Source
Total, successful and failed requests; achieved versus target rate; dropped iterations; active virtual users	k6
Latency p50, p95 and p99; threshold and check results	k6
Outbox pending count and oldest age; Kafka publish failures	Platform metrics via Prometheus
Consumer lag per topic and partition; processing rate; retries; dead-letter depth	Platform metrics via Prometheus
SMTP results by class	Platform metrics via Prometheus
CPU, memory, garbage collection, thread-pool queue, database connections	OpenTelemetry runtime metrics and cAdvisor via Prometheus

Results and regression detection

- Each run stores the test ID and scenario, start and end time, target environment, configured and achieved throughput, request totals, latency percentiles, threshold results, error breakdown, backend and Kafka measurements and final status, plus the git commit, application version, an environment fingerprint (hardware, container limits, dependency versions) and the test data size.
- Users compare runs side by side and pin one run per scenario and environment as the baseline.
- A run is flagged as a regression when p95 latency is more than 15% above baseline or sustainable capacity is more than 10% below it. Both thresholds are configurable.
- CI runs a short performance smoke test on every pull request with relative thresholds, and a nightly capacity run on a fixed machine whose results are trended in the dashboard.
- Shared CI runners are too noisy for absolute numbers. Capacity figures are quoted only from the fixed machine, always with its specifications.
- Raw per-request data is not retained. Five-second aggregates are kept for 90 days and run summaries indefinitely.

Security, privacy and compliance

Security targets OWASP ASVS level 2 for the services and the OWASP API Security Top 10 (2023) for the HTTP surface, backed by a written threat model kept in the repo.

Identity and access

- Keycloak (Apache 2.0) is the identity provider, with its realm exported as code in the repo.
- Client apps authenticate with OAuth 2.0 client credentials. Tokens are short-lived JWTs validated for issuer, audience, signature (against the provider's published keys) and expiry. Each client app maps to one platform client record, and the `notifications:send` scope is required to send.
- Dashboard users sign in with Authorization Code and PKCE. NotificationWebApi acts as the backend-for-frontend: it keeps tokens server-side and gives the browser an HttpOnly, SameSite=Strict session cookie. State-changing calls need an anti-forgery token.
- Roles are additive and enforced on the server for every endpoint; the UI only hides what the server would refuse.
- A client can read only its own notifications, through `GET /api/v1/notifications/{id}`.

Capability	Viewer	Operator	Admin	LoadTester
View notifications, metrics and load-test results (recipients masked)	Yes	Yes	Yes	Yes
Retry, replay, edit suppression list	No	Yes	Yes	No
Manage clients, templates, sender profiles; view audit log	No	No	Yes	No
Start and cancel load tests, pin baselines	No	No	No	Yes

Unmasking recipient addresses is a separate `pii:view` permission, and each use is audited.

Secrets and cryptography

- SMTP passwords use envelope encryption: a data key per secret (AES-256-GCM) wrapped by a master key from the environment or a secrets file, with a key ID stored beside the ciphertext so keys can rotate. OpenBao (MPL 2.0) is an optional upgrade.

HashiCorp Vault is excluded because its license is no longer open source.

- Client credentials live in Keycloak, not in the platform database.
- Secret fields are write-only in the API and UI; reads return only "set" or "not set".
- Recipient addresses and template data are encrypted at rest. Lookups use a keyed hash (HMAC-SHA-256 with a server-side pepper).
- Logging redacts secrets and email bodies, tests fail the build if either reaches a log line, and gitleaks scans every commit.
- Outside the development profile, Kafka traffic uses TLS and SASL/SCRAM with per-service ACLs.

Threat model highlights

Threat	Mitigation
Stolen or misused client token	Short lifetimes, per-client rate limits and quotas, revocation in Keycloak, alert on abnormal send rates
Platform abused as a spam or phishing relay	Template-only content (no free-form bodies), per-client template and sender grants, recipient caps, daily quotas, suppression list
Template injection	Sandboxed Liquid with no type access, render timeout and output cap
Header or recipient injection	Address validation, CR and LF rejection, header APIs instead of string building
SSRF through SMTP tests or load-test targets	Host allow-lists, private and link-local ranges blocked unless listed, addresses resolved and pinned at use
Broken object-level authorization	Ownership checks in the data layer and an authorization test matrix over every endpoint and role
Personal data in logs, queues or backups	Claim-check messages, redaction, encryption at rest, retention jobs
Compromised dependency or image	Pinned versions, SBOM, Trivy scans, Dependabot, license check, minimal non-root images

Threat	Mitigation
Load-test abuse	Caps, allow-list, role and audit from Load testing and capacity
Tampering with the audit trail	Insert-only database role for the audit table

HTTPS is required outside local development, with HSTS, a strict CORS allow-list, standard security headers, a strict Content-Security-Policy on the dashboard, request-size and JSON-depth limits, and no stack traces in responses.

Privacy

- Minimize: rendered email bodies are never stored. The encrypted payload is kept only as long as retries may need it.
- Mask by default in the UI and in logs, for example `j***@example.com`.
- Retention: payloads are erased 7 days after a notification reaches a final state and metadata after 90 days, both configurable (see Data model and retention).
- Erasure: an admin action removes a recipient's address and payload everywhere. The suppression list keeps only a keyed hash.
- Kafka messages carry no personal data, so topic retention does not extend personal-data retention.
- The SMTP provider is a data processor, and the README says so.

The design follows data-protection principles (minimization, purpose limitation, retention limits). It is not legal advice or a compliance certification.

Email standards

- SPF, DKIM and DMARC are configured in the sender's DNS. A sender-domain health check in the dashboard reads the records and warns when they are missing or misaligned.
- Messages follow RFC 5322 and MIME multipart/alternative.
- Subscription mail supports one-click unsubscribe (RFC 8058).
- Bounces (RFC 3464 delivery status notifications) and complaints are processed when the SMTP provider supplies them. Without that feedback, SMTP-accepted is the last state the platform can see, and the dashboard says so.
- Clients are responsible for lawful basis and consent. The platform enforces suppression and unsubscribe.

Audit

Every sensitive action writes an append-only audit event with actor, action, entity, redacted before and after values, source IP and correlation ID. Sensitive actions are changes to clients, templates, sender profiles and suppression entries, replays, unmasking, and load-test starts, cancellations and baseline changes.

Observability and SLOs

Every promise in this PRD is expressed as a service-level objective with a metric behind it, so the dashboard and the alerts read from the same numbers.

Service-level objectives

Objective	Target	Window
Send API availability (valid requests answered without a 5xx)	99.9%	30 days
Accept latency	p95 < 200 ms and p99 < 500 ms	30 days
Accept to SMTP hand-off, transactional mail	99% within 30 s	30 days
Accepted notifications lost	0	Always
Notifications reaching a final state	100% within 24 hours	Daily

A 99.9% target over 30 days allows about 43 minutes of failure. If more than half of that budget burns in one week, feature work pauses for reliability work. The policy is written in the repo.

Telemetry design

- Traces: OpenTelemetry for .NET. The W3C `traceparent` travels in Kafka message headers, so one trace spans the API, outbox, Kafka, worker and SMTP send. Export goes over OTLP to a collector and Jaeger (Apache 2.0).
- Metrics: OpenTelemetry metrics scraped by Prometheus (Apache 2.0). Custom metrics use the `notif_` prefix: `notif_accepted_total`, `notif_outbox_pending`, `notif_outbox_oldest_age_seconds`, `notif_kafka_consumer_lag`,

`notif_state_transitions_total`, `notif_send_duration_seconds`, `notif_smtp_results_total`, `notif_retries_total` and `notif_dlq_depth`. Client ID is an allowed label; recipient and notification IDs never are.

- Logs: Serilog JSON to stdout with timestamp, level, service, trace ID, correlation ID, notification ID and client ID, and no personal data. Loki is optional.
- Health: `/health/live` and `/health/ready`; readiness checks the database, Kafka and applied migrations.
- Consumer lag: the platform computes it from committed versus end offsets through the Kafka admin client, so the dashboard works without extra tooling.
- Grafana OSS (AGPL-3.0) is optional, with dashboards as code for operators. The React dashboard is the product UI.

Alerts

Alert rules live in the repo, are unit-tested with promtool, and each links to a runbook.

Alert	Condition	Runbook
Outbox stuck	Oldest unpublished row older than 60 s for 2 minutes	outbox-stuck
Lag growing	Consumer lag rising for 10 minutes	consumer-lag
Dead letters present	Dead-letter depth above 0 for 15 minutes	dead-letters
SMTP degraded	SMTP failure ratio above 5% for 5 minutes	smtp-degraded
Error-budget burn	Fast and slow burn rates on the availability objective (multi-window)	availability

Data model and retention

PostgreSQL holds all state. The notifications table is the system of record and is partitioned by month, so removing old data is a cheap partition drop rather than a slow delete.

Core tables

Table	Purpose	Notes
clients	Tenant identity	Keycloak client ID (unique), status, rate limit, daily quota, synthetic flag
client_template_grants, client_sender_grants	Which templates and sender profiles a client may use	Many-to-many
templates, template_versions	A template and its immutable versions	Subject, HTML, text, variable schema (JSON Schema), status, content hash
sender_profiles	From-address, SMTP host, port, TLS mode, send-rate cap	Credentials referenced by secret ID
secrets	Envelope-encrypted values	Key ID, ciphertext, nonce
notifications	System of record	UUIDv7 ID, client, template version, status, expires_at, recipient hash, masked recipient for display, encrypted recipients and payload, payload hash, correlation ID, idempotency key, timestamps; partitioned monthly
notification_attempts	One row per send attempt	Attempt number, start and end, outcome class, sanitized SMTP code, worker ID, duration
outbox	Messages waiting to be published	Topic, key, headers, created and published times; partial index on unpublished rows

Table	Purpose	Notes
suppressions	Addresses not to be mailed	Keyed hash, reason, source, optional expiry
audit_events	Append-only audit trail	Insert-only database role
load_test_configs, load_test_runs	Test definitions and runs	Initiator, environment, verdict, environment fingerprint, git commit
load_test_buckets	5-second aggregates per run	Rates, latency percentiles, errors, virtual users, backend gauges
load_test_baselines	Pinned baseline per scenario and environment	Reference to a run

Conventions

- UUIDv7 identifiers (time-ordered; .NET creates them with Guid.CreateVersion7), UTC timestamps with time zone, snake_case names.
- Indexes: notifications by client and created time, a partial index on non-final statuses for stalled-message queries, correlation ID, and recipient hash.
- Migrations are EF Core migrations applied by a one-shot migrator job, not at application start. They are forward-only and follow expand-then-contract, so the previous application version keeps working during a rollout.
- Integration tests run the migrations on an empty database and on a snapshot of the previous release.

Retention

All periods are configuration values. Defaults:

Data	Default retention	Mechanism
Encrypted payload and recipients	7 days after a final state	Scheduled purge sets the columns to null
Idempotency keys	24 hours	Scheduled purge
Notification and attempt metadata	90 days	Drop monthly partitions

Data	Default retention	Mechanism
Published outbox rows	24 hours	Scheduled delete
Load-test 5-second aggregates	90 days	Scheduled delete
Load-test run summaries	Indefinite	None
Audit events	1 year	Scheduled delete by an owner role
Kafka topics	3 days (dead letters 14 days)	Topic configuration

Technology stack and license audit

Every component below is free to use in a commercial product, and four popular .NET libraries are excluded because their current versions use commercial or reciprocal licenses. .NET 8 and .NET 9 both reach end of support on 10 November 2026, so if the existing code targets either one, moving to .NET 10 (LTS, supported until 14 November 2028) is the first task of Phase 0.

Stack

Layer	Choice	License
Runtime and API	.NET 10, ASP.NET Core, built-in OpenAPI generation	MIT
Messaging	Apache Kafka 4.x in KRaft mode (no ZooKeeper), Confluent.Kafka client	Apache 2.0
Database	PostgreSQL, Npgsql, EF Core	PostgreSQL License, MIT
Resilience	Polly through Microsoft.Extensions.Resilience	BSD-3-Clause, MIT
Email	MailKit, Fluid (Liquid templates), Mailpit as the test server	MIT
Identity	Keycloak	Apache 2.0

Layer	Choice	License
Secrets	AES-GCM from the .NET base library; OpenBao optional	MIT, MPL 2.0
Real time	SignalR	MIT
Mapping	Hand-written, or the Mapperly source generator	Apache 2.0
Telemetry	Serilog, OpenTelemetry, Prometheus, Alertmanager, Jaeger	Apache 2.0
Optional operator UI	Grafana OSS, Loki	AGPL-3.0
Load testing	k6 CLI as an unmodified separate process; Toxiproxy for fault injection	AGPL-3.0, MIT
Frontend	React, TypeScript, Vite, TanStack Query and Table, Recharts, Tailwind, shadcn/ui, React Hook Form, Zod	MIT
.NET testing	xUnit, Shouldly or AwesomeAssertions, NSubstitute, Testcontainers, Respawn, NetArchTest, Stryker.NET	Apache 2.0, BSD-3-Clause, MIT
Frontend testing	Vitest, React Testing Library, MSW, Playwright, axe-core	MIT, Apache 2.0, MPL 2.0
CI and security	GitHub Actions and CodeQL (free for public repositories), Trivy, gitleaks, Dependabot, CycloneDX SBOM tooling, OWASP ZAP	Open source or free tier
Containers	Docker Engine and Compose	Apache 2.0

Licenses are listed as I know them, apart from the exclusions below, which I checked against their sources. A license check in CI (for example an open-source NuGet license scanner with an allow-list of MIT, Apache-2.0, BSD, PostgreSQL and MPL-2.0) re-verifies them on every dependency change. k6 and Grafana are AGPL-3.0: run them unmodified as separate programs and say so in the README. This is not legal advice.

Excluded libraries

Library	Why	Use instead
MediatR 13 and later	Dual-licensed under the Reciprocal Public License 1.5 or a commercial license; the free Community edition covers individuals and companies under US$5 million in revenue but still requires registering a key	Plain handler classes
AutoMapper 15 and later	Same dual license as MediatR	Mapperly or hand-written mapping
MassTransit 9	NuGet describes it as a commercial product that must be licensed; earlier versions remain open source	Confluent.Kafka directly
FluentAssertions 8 and later	Moved to the Xceed license, reported at about US$130 per developer per year for commercial use; 7.x remains Apache 2.0	Shouldly or AwesomeAssertions
HashiCorp Vault	Business Source License, which is not open source	OpenBao
Confluent Schema Registry	Confluent Community License, which is not open source	JSON Schema contracts in the shared library, or Apicurio Registry (Apache 2.0) if a registry is wanted

A personal project would qualify for the MediatR and AutoMapper free tier. Avoiding them still keeps the "completely free tools" claim simple and removes a question a reviewer might ask.

Sources, checked 9 October 2026

- .NET and .NET Core support policy
- AutoMapper and MediatR commercial editions launch today
- MassTransit 9.1.1 on NuGet

- Fluent Assertions license change (DevClass)

Engineering standards

Quality gates are automated, so the repository proves its own standards on every pull request.

Code and architecture

- Each service has four layers (domain, application, infrastructure, API) with dependencies pointing inward. Architecture tests (NetArchTest) fail the build if a layer reaches the wrong way or if the shared library references application code.
- C# nullable reference types on, warnings treated as errors, Roslyn analyzers with a shared .editorconfig, central package management, and format checks in CI.
- TypeScript in strict mode, with ESLint and Prettier.
- Conventional Commits, semantic versioning, and short-lived branches into a protected main branch.

Test strategy

Level	Scope	Tools	Gate
Unit	State machine, retry classification, idempotency rules, rate-limit policy, template rendering, validation	xUnit, Shouldly, NSubstitute	80% line coverage on domain and application code; Stryker mutation testing on the state machine and retry logic
Integration	Outbox relay, consumer commit behavior, migrations, delivery to Mailpit, Keycloak token validation	Testcontainers with PostgreSQL, Kafka, Mailpit and Keycloak	All pass on every pull request
Contract	OpenAPI document and the generated TypeScript client	oasdiff, TypeScript compiler	A breaking change without a version bump fails the build

Level	Scope	Tools	Gate
Security	Authorization matrix across every endpoint and role, object-level access, header injection, SSRF cases, dynamic scan	xUnit, OWASP ZAP baseline	No high findings
Resilience	Every failure listed under Reliability and data consistency	Testcontainers, Toxiproxy, container kill	No loss, bounded duplicates, correct final state
End to end	Sign in, create a client and template, send through the API, see the notification in the dashboard, run a smoke load test	Playwright against the compose stack	Happy path passes in CI
Frontend	Components, forms, charts, accessibility	Vitest, React Testing Library, MSW, axe-core	No serious accessibility violations
Performance	Smoke test on every pull request, capacity run nightly	k6 through LoadTestWorker	Relative regression thresholds

Pipeline

1. Build, lint and format check.
2. Unit tests.
3. Integration and contract tests.
4. Build container images.
5. Security: CodeQL, Trivy on files and images, gitleaks, license check, SBOM.
6. End-to-end tests and the ZAP baseline scan against the compose stack.
7. On a version tag: publish images to GitHub Container Registry and create release notes.

Documentation as code

- README, and an ARCHITECTURE file with C4 diagrams written in Mermaid.

- One ADR per decision in Key decisions, in `docs/adr`.
- Runbooks in `docs/runbooks` and the threat model in `docs/security`.
- API reference generated from the OpenAPI document and served with Scalar or Swagger UI.
- Standards followed and named in the docs: twelve-factor configuration, RFC 9457, semantic versioning, OWASP ASVS, WCAG 2.2 and OpenTelemetry semantic conventions.

Deployment and operations

The whole platform starts with one command, and the same images run unchanged in a container orchestrator.

Docker Compose profiles

Profile	Services
core (default)	PostgreSQL, Kafka (KRaft, single node), Keycloak, NotificationWebApi, NotificationSenderApi, NotificationWorker, dashboard, Mailpit, plus one-shot migrator and topic-provisioning jobs
observability	OpenTelemetry collector, Prometheus, Alertmanager, Jaeger, optional Grafana and Loki
loadtest	LoadTestWorker with k6, Toxiproxy, cAdvisor
demo	Seeder that creates demo clients, templates and 30 days of synthetic history so the dashboard is populated on first launch

`docker compose up` starts core; other profiles are added with `--profile`. Running every profile needs several gigabytes of memory, mostly for Kafka, Keycloak and PostgreSQL. Measure the figure and document it as the minimum hardware.

Environments

- Development: Compose on a laptop, plaintext Kafka on an isolated network, replication factor 1.
- CI: an ephemeral Compose stack per pipeline run.

- Staging or demo: optionally one small server running Compose. It is the environment on the load-test allow-list.
- Production-like profile: Kafka replication factor 3 with min.insync.replicas of 2, TLS and SASL, and secrets from files or a secrets manager. A Helm chart or Kustomize overlay with KEDA scaling consumers on lag is a stretch goal.

Configuration and startup

- Configuration comes from environment variables and secret files, with a documented `.env.example`. Services validate it at start and fail fast.
- Migrations and Kafka topics are created by one-shot jobs before the services start.
- Every container has CPU and memory limits, including heap limits for Kafka and Keycloak.
- Releases are tagged with semantic versions. Rollback redeploys the previous tag, which works because migrations stay backward compatible for one version.

Backup and recovery

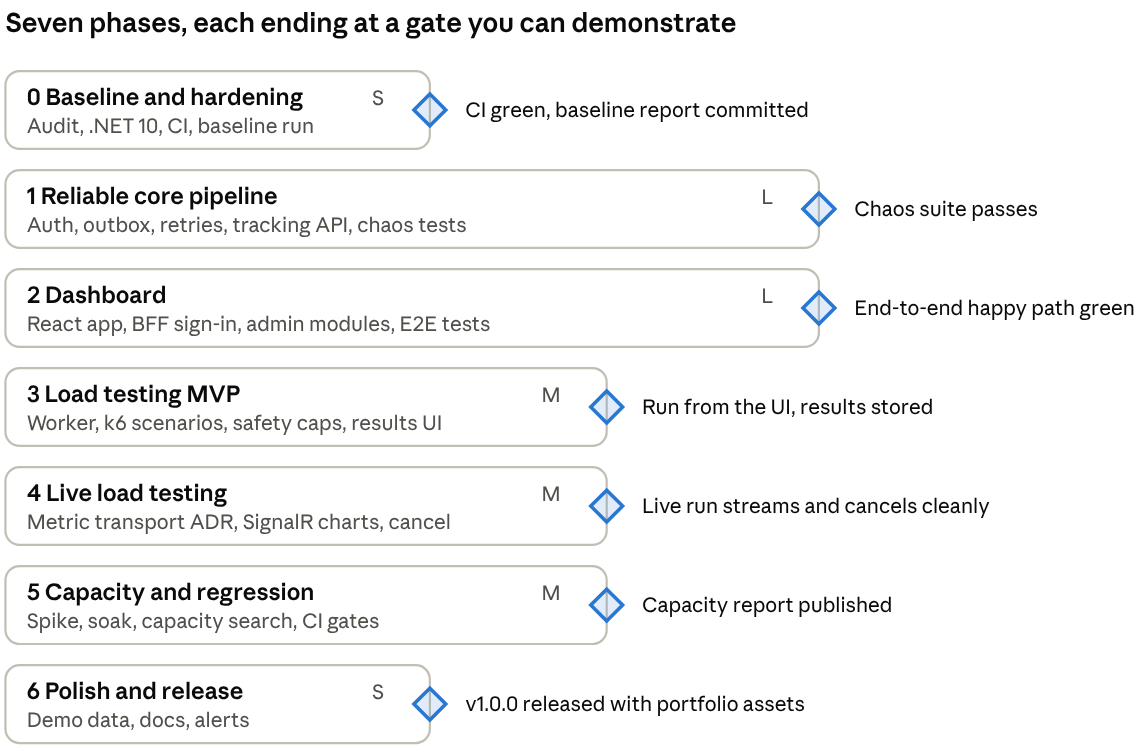
- PostgreSQL base backups plus WAL archiving (for example pgBackRest, MIT) in the production-like setup. Targets: recovery point 15 minutes, recovery time 1 hour. A scripted restore drill proves that a backup restores and passes data checks.
- Kafka is not the system of record. After broker data loss, a re-drive job republishes every notification that is not in a final state from PostgreSQL.

Runbooks

Outbox stuck, consumer lag, dead letters, SMTP degraded, availability, rotate SMTP credentials, rotate the master key, Kafka re-drive, stuck load-test run, restore from backup.

Roadmap

Seven phases take the project from a measured baseline to a published capacity report, each ending at a gate that can be demonstrated. Phase 0 is new: it measures the existing code first, which turns the final numbers into a before-and-after story instead of a claim.



Diamond = exit gate. Band length = relative size: S 1 to 2 weeks, M 2 to 4, L 4 to 6.

roadmap · 7 phases, 7 gates

Sizes are rough, for one developer working part time, and the plan totals about 16 to 28 weeks. Recalibrate after Phase 0, when the state of the existing code is known.

What each phase contains

Phase	Work
0 Baseline and hardening	Gap table of the existing code against this PRD; upgrade to .NET 10 if needed; characterization tests around current behavior; CI skeleton; Compose core profile; k6 baseline run of the current API; first ADRs
1 Reliable core pipeline	Keycloak authentication; send endpoint with idempotency and problem details; outbox and relay; worker with retry topics and dead letters; state machine; MailKit with Mailpit; template and sender-profile APIs; notification tracking API; OpenTelemetry baseline; chaos tests for the core failures

Phase	Work
2 Dashboard	React app with backend-for-frontend sign-in; overview, notification explorer, clients, templates, sender profiles, suppression list, audit log; generated API client; accessibility checks; Playwright end-to-end test
3 Load testing MVP	LoadTestWorker; versioned k6 scenarios for smoke, load and stress; safety controls and caps; persisted results; completed-run pages in the dashboard
4 Live load testing	Metric transport spike and ADR; SignalR streaming; live charts; cancellation; concurrency control; threshold verdicts
5 Capacity and regression	Spike, soak and capacity-search scenarios; backend instrumentation for lag and processing rate; baselines and regression flags; pull-request performance smoke test; nightly capacity run; published capacity report
6 Polish and release	Seeded demo, documentation and runbooks; alert rules with tests; security scan clean-up; v1.0.0 tag and portfolio assets

Priorities (MoSCoW)

Priority	Scope
Must	Outbox and idempotent accept; retries and dead letters; authentication and roles; notification tracking; overview and explorer; k6 load and stress tests with stored results; capacity report; CI with tests; one-command Compose run
Should	Spike and soak tests; live streaming; regression baselines; suppression list; SLO alerts; audit log; template versioning; ZAP scan
Could	Kubernetes with KEDA; OpenBao; priority topic; client webhooks; bounce processing; Apicurio registry; Grafana dashboards

Priority	Scope
Won't in v2.0	Marketing campaigns; attachments; inbound email; SMS and push; multi-region; drag-and-drop template editor; load tests with real email

Definition of done

- [ ] Authorized client apps can send email through the public API, and a repeated Idempotency-Key never creates a second notification.
- [ ] Accepted notifications are durable in PostgreSQL before 202, published to Kafka by the outbox relay, and processed asynchronously.
- [ ] Failures, retries, dead letters and outcomes can be inspected in the dashboard, which shows HTTP acceptance, SMTP acceptance and delivery as distinct outcomes.
- [ ] Administrators can manage clients, templates and sender profiles, and every sensitive change is audited.
- [ ] The dashboard shows notification analytics from real data.
- [ ] An authorized user can start, watch, cancel and compare load tests. k6 runs separately from the API and sends real traffic to a configured, allow-listed environment.
- [ ] The published capacity report states sustainable acceptance and processing rates, headroom, bottleneck, method and hardware.
- [ ] The chaos suite passes: no accepted notification lost, duplicates bounded and measured.
- [ ] Authentication, authorization, rate limits, resource caps and environment restrictions protect the API and the load-test subsystem.
- [ ] SLOs, alert rules and runbooks exist, and the observability profile shows them working.
- [ ] CI runs unit, integration, contract, security, resilience and end-to-end tests, and a license check confirms every dependency is free to use.
- [ ] The whole stack runs from one command with no paid service.

Portfolio packaging

A reviewer skims a repository before reading it, so the project is packaged to prove quality quickly: one command to run, one page of results, and evidence behind every claim.

Repository checklist

- [ ] README with a one-paragraph pitch, architecture diagram, short demo video or GIF, the one-command quickstart, and links to the results below
- [ ] Before-and-after performance story: the Phase 0 baseline of the existing code next to the final capacity report, in `docs/performance.md`, with hardware specifications, k6 scripts and raw summaries
- [ ] Chaos report: each failure listed under Reliability and data consistency, with its test result
- [ ] At least ten ADRs, the threat model, the SLO document and the runbooks
- [ ] Seeded demo data and documented demo credentials, so the dashboard is alive on first launch
- [ ] Dashboard screenshots: overview, notification timeline, live load-test view, capacity report
- [ ] CI badges for build, tests, coverage, CodeQL and license check
- [ ] LICENSE (MIT or Apache 2.0), SECURITY.md, CONTRIBUTING.md and a CHANGELOG
- [ ] Tagged releases with container images on GitHub Container Registry
- [ ] A short write-up on the outbox, the retry design and how capacity is defined

Interview talking points

Prepare an answer for each of these: why an outbox instead of publishing from the API; why at-least-once and not exactly-once; why open-model load generation; how capacity differs from peak throughput; what the baseline showed and what changed; and what you would change at ten times the load (more partitions, per-tenant isolation, a dedicated SMTP pool, autoscaling on lag).

Product feel

- Give it a real product name, a simple logo and a consistent dashboard theme. Check that the name is not an existing product.
- Kafka and Keycloak are heavy for free hosting tiers, so the demo is a recorded walkthrough plus the one-command local run. A single small server is optional and is the only possible cost.

Risks and open questions

The largest risk is scope for a single maintainer, which the phase plan and priorities are built to contain.

Risk	Mitigation
Scope is too large for one maintainer	MoSCoW priorities, vertical slices, and each phase ending in something demonstrable
The existing code diverges from this design	Phase 0 audit, characterization tests before refactoring, incremental migration
Load-test numbers are meaningless on a laptop	One fixed machine for capacity runs, published specifications, relative regression thresholds, no cross-machine comparisons
k6 metric transport adds overhead or loses accuracy	Phase 3 spike with explicit criteria, recorded as an ADR
A free library changes its license	CI license check, pinned versions, one ADR per dependency choice
Kafka and Keycloak are heavy on a laptop	Compose profiles, memory limits, documented minimum hardware
Over-engineering	Every feature traces to a requirement or a metric, and the out-of-scope list is enforced

Questions to answer before Phase 1 starts

1. Which .NET version, database and authentication does the existing code use? This sets the size of Phase 0.
2. Do existing callers of `POST /api/v1.1/Email/SendEmail` send an idempotency key today? If not, should the key be optional during a migration window?
3. Do you want real delivery feedback such as bounces? That needs an SMTP provider with webhooks or a mailbox for delivery status notifications. Otherwise v2.0 treats SMTP-accepted as the last visible state.
4. Which machine will run the capacity tests, and will you quote its specifications in the portfolio?
5. Will the repository be public? Free CI minutes and CodeQL depend on it.